

I have a lab assignment I need to do and I need help understanding the concepts, dont give me the answers straight away

Sounds good — I'll work with you to build understanding rather than just hand over answers. What's the lab assignment about? Feel free to share the questions or topics, and let me know what course it's for (database, Java, linear algebra, etc.) so I can pitch explanations at the right level.

Its based on uninformed and informed search, the lab is based on a drone using these different algorithms

Nice, this lines up with the AIMA-style search work you've done before. Let's build this up piece by piece.

First, let's make sure the foundation is solid before touching drone-specific stuff. A few questions to get oriented:

1. What does the drone's environment look like? Is it a grid (like a 2D/3D grid of cells), a graph with named locations, or something else? This matters a lot because it determines how you'll represent states.

2. Which algorithms does the lab want you to implement? Likely candidates: BFS, DFS, maybe DLS/IDS (uninformed), and then A* and/or Greedy Best-First Search (informed) — since you mentioned "informed search" too.

3. What's the drone's goal? Eg., reach a target cell, find a charging station, avoid obstacles, minimize fuel/distance, etc

Once I know that, we can start with the core conceptual distinction that the whole lab probably hinges on:

Uninformed vs Informed search — the key difference isn't "good vs bad," it's about what information the algorithm has access to:

• Uninformed (BFS, DFS, IDS, UCS): the algorithm only knows the structure of the problem — states, actions, and which states are goals. It has no sense of how "close" it is to the goal.

• Informed (Greedy, A*): the algorithm additionally has a **heuristic function** $h(n)$ — an estimate of distance/cost from a node to the goal. This lets it prioritize promising paths instead of blindly exploring.

Does that distinction make sense so far? And can you tell me more about the drone setup (grid/graph, what counts as a "move," and whether there's a heuristic like Euclidean/Manhattan distance mentioned in the assignment)?

Its a 2d grid and all the algorithms were used, the drones goal is a target cell

Good, that's a clean setup. Let's build the mental model piece by piece.

1. State representation

In a 2D grid, what do you think a "state" should be? Think about what the drone needs to know at any point in time to decide its next move.

(Hint: it's probably just one thing — but it's worth you saying it explicitly, because people sometimes overcomplicate this.)

2. Actions / successors

From a given cell, what are the possible moves? Typically it's up/down/left/right (4 directions), sometimes diagonals too (8 directions). Does your lab grid allow diagonal movement, or just the 4 cardinal directions? And are there obstacles/walls in the grid that block certain moves?

3. The frontier — this is where the algorithms actually differ

Here's the core idea I want you to sit with: **every one of these search algorithms is the same basic loop** —

```
Code
put start node in frontier
while frontier is not empty:
  pick a node from frontier (HOW you pick is what makes algorithms different)
  if it's the goal: return the path
  otherwise, expand it (find its neighbors) and add them to frontier
```

The entire difference between BFS, DFS, DLS, IDS, Greedy, and A* comes down to two things:

1. What data structure is the "frontier" (queue, stack, priority queue)?

2. What rule decides which node to pop next?

So here's a question for you: do you remember, from your earlier search lab, what data structure BFS uses for its frontier, and what DFS uses? Why do those choices produce "breadth-first" vs "depth-first" behavior?

so what data structures do they use and why

Let's work through each one and why the structure produces that behavior — not just memorize "BFS = queue."

BFS uses a queue (FIFO — First In, First Out)

Think about what FIFO means: whatever goes in *first* comes out *first*. So if you put the start node's neighbors in the queue, then *their* neighbors go in *after* them. When you pop from the front, you're always popping the oldest-added nodes first — meaning you fully explore everything at distance 1 from start before you even look at anything at distance 2.

That's literally what "breadth" means here: expanding outward in layers, like ripples in a pond.

DFS uses a stack (LIFO — Last In, First Out)

Now flip it: the *last* thing you added is the *first* thing you pop. So if you expand a node and push its 4 neighbors, the next thing you pop is one of those neighbors (recently pushed) — not anything left over from earlier layers. You then immediately expand that neighbor, push its neighbors, and pop one of those next.

So DFS commits to a path and goes deep, deep, deep, before it ever backs up and tries a sibling branch. That's the "depth" in depth-first.

Quick check — based on that logic, can you predict: if a drone is on a grid with no obstacles and the goal is 3 cells away, would BFS or DFS find the *shortest* path reliably? Why might the other one not?

(Try answering before I confirm — this is the kind of thing labs love to test.)

I think bfs

Exactly right, and you should be confident about why.

BFS explores layer by layer — everything 1 step away, then everything 2 steps away, then 3 steps away. So the very first time it reaches the goal, it's guaranteed to be via the shortest possible path (in terms of number of moves), because there's no way a longer path could have been discovered first.

DFS, on the other hand, just commits to one direction and keeps going deep. It might stumble into the goal immediately if it gets lucky, but it might also wander far away first, taking a long, winding 15-move path to a goal that was only 3 moves away. It'll still *find* the goal eventually (on a finite grid), but there's no guarantee it's the shortest path.

This is a really important general principle worth holding onto: **BFS guarantees shortest path (in steps) when all moves cost the same**. DFS does not.

Now — here's where it gets more interesting. DLS and IDS are both *variants* of DFS. Do you know what DLS (Depth-Limited Search) changes about plain DFS, and why that change exists? Think about what problem plain DFS has on an infinite or very large grid.

why does dfs use a stack

Good question — let's think about it from the *goal* backwards, rather than just memorizing it.

The defining feature of DFS is: **once you start down a path, you commit to following it all the way before trying anything else**. So whatever data structure DFS uses needs to naturally produce that "go deep immediately" behavior.

Walk through it with a tiny example. Say the drone is at cell A, and A has neighbors B and C.

You expand A, and push B and C onto the stack. Stack now (top to bottom): [C, B]

You pop the **top** of the stack — that's C (whatever you pushed *last*). You expand C.

Say C has a neighbor D. You push D. Stack: [D, B]

You pop D next — not B! Even though B has been sitting there waiting longer.

See what happened? Because a stack always gives you back the most *recently* added item, the algorithm naturally chains forward: expand a node → push its children → immediately pop one of those children (not anything older) → expand it → push its children → pop one of those next...

It never "goes back" to B until the entire C-branch is exhausted (no more children to explore, dead end, or stack empties out down to B). That's exactly depth-first behavior — you tunnel down one branch fully before backtracking.

Now contrast that with a queue: If you used a queue instead, after expanding A and adding B and C, you'd pop B next (oldest in line) — not D. That would force the algorithm to spread out across the same "depth level" before going deeper, which is the *opposite* of what DFS is supposed to do.

So the stack isn't an arbitrary implementation detail — it's the *mechanism* that makes "always go deeper before going wider" actually happen.

Does that click? Want to try tracing through it yourself with a slightly bigger example, like a 3x3 grid?

yes

Let's set up a small grid and you trace it yourself.

The grid (3x3), rows 0-2, columns 0-2:

```
Code
(0,0) (0,1) (0,2)
(1,0) (1,1) (1,2)
(2,0) (2,1) (2,2)
```

Drone starts at (0,0). Goal is (2,2). Moves allowed: **right and down only** (let's simplify — no up/left, no diagonals), just to keep the branching factor small while you get the hang of it. No obstacles.

So from any cell, the neighbor order we'll push is: **first push "down", then push "right"** (you'll see why order matters in a second).

Your task: trace DFS using a stack. I'll start you off, you continue.

Stack: [(0, 0)]

Pop (0,0). Not goal. Expand it: neighbors are down=(1,0), right=(0,1). Push down first, then right.

Stack: [(1, 0), (0, 1)]

Pop the **top** of the stack — which one is that?

Go ahead from here: tell me which node gets popped next, expand it (give its down/right neighbors), push them in the same order (down then right), and show me the new stack. Keep going for a few steps and tell me where you end up — does it tunnel straight down, straight right, or some mix? Try it and show me your steps.

could you tell me

I get the temptation to just see it, but you'll learn this way faster by running it yourself — it's only a few steps and you already have the rule down (pop the top, expand, push down-then-right).

Let's lower the friction instead. You already correctly identified the stack from my last step: **Stack: [(1, 0), (0, 1)]**

Which one is on **top**? (Top = the one that was pushed *last*, i.e., the rightmost/last one in that list.)

Just answer that one small question, and I'll help you do the expand-and-push step together so you're not stuck on the mechanics.

(1,1)

Not quite — let's slow down on just this one step.

The stack right now is: [(1, 0), (0, 1)]

We pushed (1, 0) first, then (0, 1) second. A stack pops whatever was pushed **last** — so the "top" is the one that went in most recently.

Between (1, 0) and (0, 1), which one did we push *second* (most recently)?

(0,1)

Exactly right. So we pop (0,1) next.

Now let's expand it:

Is (0,1) the goal (2,2)? No.

What are its neighbors? Remember: down and right only.

Down from (0,1) → row+1, same column → (1,1)

Right from (0,1) → same row, column+1 → (0,2)

We push down first, then right (same rule as before).

Stack before this step: [(1, 0)] (we removed (0,1) when we popped it)

Now push (1,1), then (0,2):

Stack: [(1, 0), (1, 1), (0, 2)]

Quick check before we continue — which one is on top now, and why?

(0,2)

Exactly — (0,2) is on top, because it was pushed last.

Notice the pattern forming: instead of going back to (1, 0) (which has been waiting since the very first expansion), the stack keeps feeding us the most recently discovered node. We're tunneling along the "right" edge of the grid instead of spreading out.

Let's continue. Pop (0,2):

Is (0,2) the goal (2,2)? No.

Neighbors of (0,2): down → (1,2). Right → would be (0,3), which is **off the grid**, so it doesn't exist — skip it.

So we only push one node this time: (1,2).

Stack before: [(1, 0), (1, 1)]

Push (1,2):

Stack: [(1, 0), (1, 1), (1, 2)]

Your turn: which node is on top now, and what are its down/right neighbors? Go ahead and do the pop + expand + push step yourself.

wait for dfs why do we avoid adding a child if its state already appears on the current path from the root to the current node

Good question — this is about **avoiding infinite loops / cycles**, and it's a subtly different concern from the depth limit itself. Let's unpack it.

Think about why DLS exists in the first place: plain DFS can run forever on an infinite (or cyclic) graph because it just keeps tunneling deeper with no stopping condition. DLS fixes that by adding a depth limit — once you hit depth L , you stop expanding, even if you haven't found the goal yet.

Now, the "don't revisit a state already on the current path" rule is a **separate but related** problem: cycles.

Picture this on your drone grid — suppose at some point we *did* allow up/left moves too (not just down/right). Then from (1,1), the drone could move right to (1,2), and from (1,2) it could move up to (0,2), and from (0,2) it could move down... back to (1,2)? Or further manipulation could get you back to a node you've already visited *on the same path*.

If that happens, your "depth" counter keeps increasing (you're adding nodes to the path) but you're not actually making any new progress — you're just walking in circles, using up your depth limit on a loop instead of genuinely exploring new territory.

So the rule is: **before adding a child to the path, check if that state is already an ancestor of the current node (i.e., already sitting somewhere on the path from root to here)**. If it is, don't add it — because revisiting it can't possibly help; you'd just be looping back into territory you're already inside of.

Quick question to check your intuition: in our simplified version of the drone problem (only down/right moves allowed, no up/left), is it actually *possible* to ever revisit a state that's already on your current path? Think about what direction the coordinates move in.

I dont think its possible

Exactly right — and you should be able to say why, not just sense it.

With only down/right moves, every step **strictly increases** either the row or the column (or both, if diagonals were allowed, but here it's one or the other). Since coordinates only ever go up, you can never land back on a cell you've already visited — that would require a coordinate to *decrease* at some point, which isn't an allowed move.

So in this restricted version of the problem, the grid is what's called a **DAG (Directed Acyclic)** graph, which means it's **acyclic** — no cycles possible.

This is a

It never "goes back" to B until the entire C-branch is exhausted (no more children to explore, dead end, or stack empties out down to B). That's exactly depth-first behavior — you tunnel down one branch fully before backtracking.

Now contrast that with a queue: If you used a queue instead, after expanding A and adding B and C, you'd pop B next (oldest in line) — not D. That would force the algorithm to spread out across the same "depth level" before going deeper, which is the *opposite* of what DFS is supposed to do.

So the stack isn't an arbitrary implementation detail — it's the *mechanism* that makes "always go deeper before going wider" actually happen.

Does that click? Want to try tracing through it yourself with a slightly bigger example, like a 3x3 grid?

yes

Let's set up a small grid and you trace it yourself.

The grid (3x3), rows 0-2, columns 0-2:

```
Code
(0,0) (0,1) (0,2)
(1,0) (1,1) (1,2)
(2,0) (2,1) (2,2)
```

Drone starts at (0,0). Goal is (2,2). Moves allowed: **right and down only** (let's simplify — no up/left, no diagonals), just to keep the branching factor small while you get the hang of it. No obstacles.

So from any cell, the neighbor order we'll push is: **first push "down", then push "right"** (you'll see why order matters in a second).

Your task: trace DFS using a stack. I'll start you off, you continue.

Stack: [(0, 0)]

Pop (0,0). Not goal. Expand it: neighbors are down=(1,0), right=(0,1). Push down first, then right.

Stack: [(1, 0), (0, 1)]

Pop the **top** of the stack — which one is that?

Go ahead from here: tell me which node gets popped next, expand it (give its down/right neighbors), push them in the same order (down then right), and show me the new stack. Keep going for a few steps and tell me where you end up — does it tunnel straight down, straight right, or some mix? Try it and show me your steps.

could you tell me

I get the temptation to just see it, but you'll learn this way faster by running it yourself — it's only a few steps and you already have the rule down (pop the top, expand, push down-then-right).

Let's lower the friction instead. You already correctly identified the stack from my last step: **Stack: [(1, 0), (0, 1)]**

Which one is on **top**? (Top = the one that was pushed *last*, i.e., the rightmost/last one in that list.)

Just answer that one small question, and I'll help you do the expand-and-push step together so you're not stuck on the mechanics.

(1,1)

Not quite — let's slow down on just this one step.

The stack right now is: [(1, 0), (0, 1)]

We pushed (1, 0) first, then (0, 1) second. A stack pops whatever was pushed **last** — so the "top" is the one that went in most recently.

Between (1, 0) and (0, 1), which one did we push *second* (most recently)?

(0,1)

Exactly right. So we pop (0,1) next.

Now let's expand it:

Is (0,1) the goal (2,2)? No.

What are its neighbors? Remember: down and right only.

Down from (0,1) → row+1, same column → (1,1)

Right from (0,1) → same row, column+1 → (0,2)

We push down first, then right (same rule as before).

Stack before this step: [(1, 0)] (we removed (0,1) when we popped it)

Now push (1,1), then (0,2):

Stack: [(1, 0), (1, 1), (0, 2)]

Quick check before we continue — which one is on top now, and why?

(0,2)

Exactly — (0,2) is on top, because it was pushed last.

Notice the pattern forming: instead of going back to (1, 0) (which has been waiting since the very first expansion), the stack keeps feeding us the most recently discovered node. We're tunneling along the "right" edge of the grid instead of spreading out.

Let's continue. Pop (0,2):

Is (0,2) the goal (2,2)? No.

Neighbors of (0,2): down → (1,2). Right → would be (0,3), which is **off the grid**, so it doesn't exist — skip it.

So we only push one node this time: (1,2).

Stack before: [(1, 0), (1, 1)]

Push (1,2):

Stack: [(1, 0), (1, 1), (1, 2)]

Your turn: which node is on top now, and what are its down/right neighbors? Go ahead and do the pop + expand + push step yourself.

wait for dfs why do we avoid adding a child if its state already appears on the current path from the root to the current node

Good question — this is about **avoiding infinite loops / cycles**, and it's a subtly different concern from the depth limit itself. Let's unpack it.

Think about why DLS exists in the first place: plain DFS can run forever on an infinite (or cyclic) graph because it just keeps tunneling deeper with no stopping condition. DLS fixes that by adding a depth limit — once you hit depth L , you stop expanding, even if you haven't found the goal yet.

Now, the "don't revisit a state already on the current path" rule is a **separate but related** problem: cycles.

Picture this on your drone grid — suppose at some point we *did* allow up/left moves too (not just down/right). Then from (1,1), the drone could move right to (1,2), and from (1,2) it could move up to (0,2), and from (0,2) it could move down... back to (1,2)? Or further manipulation could get you back to a node you've already visited *on the same path*.

If that happens, your "depth" counter keeps increasing (you're adding nodes to the path) but you're not actually making any new progress — you're just walking in circles, using up your depth limit on a loop instead of genuinely exploring new territory.

So the rule is: **before adding a child to the path, check if that state is already an ancestor of the current node (i.e., already sitting somewhere on the path from root to here)**. If it is, don't add it — because revisiting it can't possibly help; you'd just be looping back into territory you're already inside of.

Quick question to check your intuition: in our simplified version of the drone problem (only down/right moves allowed, no up/left), is it actually *possible* to ever revisit a state that's already on your current path? Think about what direction the coordinates move in.

I dont think its possible

Exactly right — and you should be able to say why, not just sense it.

With only down/right moves, every step **strictly increases** either the row or the column (or both, if diagonals were allowed, but here it's one or the other). Since coordinates only ever go up, you can never land back on a cell you've already visited — that would require a coordinate to *decrease* at some point, which isn't an allowed move.

So in this restricted version of the problem, the grid is what's called a **DAG (Directed Acyclic)** graph, which means it's **acyclic** — no cycles possible.

This is a

22:40 Sat 20 Jun🔍📶🔊🔇

Exactly right — and you should be able to say why, not just sense it.

With only down/right moves, every step **strictly increases** either the row or the column (or both, if diagonals were allowed, but here it's one or the other). Since coordinates only ever go up, you can never land back on a cell you've already visited — that would require a coordinate to *decrease* at some point, which isn't an allowed move.

So in this restricted version of the problem, the grid is what's called a **DAG (Directed Acyclic Graph)** along the path — no cycles are possible by construction. That's exactly why the cycle-check in DLS would never actually trigger here: it's a safeguard for *general* graphs where backtracking moves (up/left, or arbitrary graph edges) could create loops.

This is a good thing to flag in your lab write-up if it's relevant: the **necessity of the "already on path" check depends on whether the action set allows revisiting states**. If your actual drone lab allows all 4 directions (up/down/left/right), then yes — cycles become possible, and that check becomes essential. Does your lab restrict movement to 4 directions, or did I oversimplify by cutting it to 2?

👍👎👏👊

now lets move to ids

Good — and since you already get DLS, IDS is going to feel almost too simple, because it's really just "DLS, but smarter about choosing the limit."

The problem IDS solves:

With DLS, you have to *pick* a depth limit *L* ahead of time. But what if you pick wrong?

- Too small → you might cut off the search before ever reaching the goal, even though a solution exists.
- Too large → you waste time exploring way deeper than needed, and you lose DLS's memory advantage by potentially storing a huge stack.

So the question becomes: how do you pick the "right" *L* without knowing the answer in advance?

IDS's answer: don't pick just one *L*. Try ***L* = 0**, then ***L* = 1**, then ***L* = 2**, then ***L* = 3**..., running a full DLS at each depth, stopping as soon as one of them finds the goal.

So conceptually:

Code

```
for L = 0, 1, 2, 3, ...:
    result = DLS(start, goal, limit=L)
    if result found:
        return result
```

Here's a question to test your intuition, since you already nulled the BFS-vs-DPS shortest path question: **does IDS guarantee finding the shortest path** (in terms of number of moves), like BFS does? Think about *why* — what happens at the moment IDS first succeeds?

👍👎👏👊

I think when ids first succeeds it stops but it might not guaratee finding the shortest path

Let's test that against the mechanics, because your instinct to be cautious is good — but here it actually works out the *other* way. Let's see why.

Think about what happens at each round:

- **L=0**: IDS only checks the start node. If it's not the goal, fail.
- **L=1**: IDS does a DFS but cuts off at depth 1. It can only reach nodes that are 1 step away from start. If the goal isn't among those, fail.
- **L=2**: Now it can reach nodes up to 2 steps away. Still bounded.
- **L=3**: Up to 3 steps away.
- ...and so on.

Notice: **IDS never even attempts to look at a node 4 steps away until it has exhausted depths 0, 1, 2, and 3 first**. The depth limit acts as a hard ceiling each round.

So here's the key question — If the goal happens to be exactly 3 steps away from the start, is there *any* way IDS could find it during the L=0, 1, or 2 rounds? And if it can't be found that early, what's the *earliest* round it could be found in?

👍👎👏👊

wait to how are ids and dls related

Good question to pause on — let's nail the relationship precisely.

DLS is a single search run with one fixed depth limit. It's a complete algorithm by itself — you give it a limit *L*, it does a depth-first search but refuses to expand any node deeper than *L*, and it returns either "found the goal" or "no solution within this limit" (which might be a false negative if the goal exists but is deeper than *L*).

IDS is DLS used as a building block, called repeatedly with increasing limits.

Code

```
IDS(start, goal):
    for L = 0, 1, 2, 3, ...:
        result = DLS(start, goal, limit=L)
        if result != "no solution within this limit":
            return result
```

So IDS doesn't replace DLS — it **wraps** DLS in a loop, re-running it from scratch each time with one more unit of depth allowed. Each individual call to DLS is oblivious to the fact that it's being called repeatedly; it just does its job: "search depth-first, don't go past *L*."

A useful way to picture it: imagine DLS is a flashlight with adjustable beam length. IDS is the process of starting with a very short beam (*L*=0), seeing nothing, then incrementally extending the beam length and re-scanning the *entire room from scratch* each time, until the beam finally reaches far enough to spot the goal.

That "from scratch each time" part is important and often surprises people — yes, IDS re-does a lot of redundant work (re-visiting shallow nodes over and over at every iteration). That seems wasteful, but it turns out to be a worthwhile trade-off: Do you want to think through why that redundant work doesn't end up being too costly compared to, say, BFS?

👍👎👏👊

moving to informed search, what is the alms aima expand

Let's connect this to something you already know, since you've worked in the AIMA framework before.

In AIMA's pseudocode kind, **EXPAND** is the function that takes a node and a problem, and generates all of that node's **children** — i.e., the new nodes you get by trying every legal action from that state.

Conceptually, here's what it does, step by step:

1. Take the current node's **state** (e.g., a cell like (1,1) on your grid).
2. Look up the problem's **ACTIONS(state)** — what moves are legal from here (e.g., up/down/left/right, minus anything blocked by walls or grid edges).
3. For each legal action, compute the **RESULT(state, action)** — the new state you'd land in if you took that action (e.g., action="down" → new state (2,1)).
4. Build a new **node** for that resulting state. This node needs to carry some bookkeeping:
 - A reference to its **parent** (so you can reconstruct the path later by walking back up)
 - The **action** that was taken to get here
 - The accumulated **path cost** $g(n)$ — cost from the *start* to this node (parent's cost + cost of this one step)

So **EXPAND** is essentially: "generate the full flat of children of this node, each one properly linked back to its parent, with cost tracked."

Here's the conceptual leap to informed search: In **uninformed** search (BFS/DFS/DLS/IDS), **EXPAND** is all you need — once a node is generated, you just dump it into the queue/stack and let the data structure's pop-order decide what happens next.

In **informed** search (Greedy, A*), **EXPAND** does the *exact same job* of generating children — but now, once a child node exists, something extra happens to it before it goes into the frontier. Do you remember what that "something extra" is, from your earlier coursework or readings?

👍👎👏👊

what is terrain cost

Good question — and it's actually a nice bridge from the "all moves cost the same" assumption we've been using up to now.

So far we've been quietly assuming every move on the grid costs exactly **1** — one step is one step, whether you go down or right. That's why BFS could guarantee shortest path: with uniform costs, "fewest steps" and "lowest total cost" are the same thing.

Terrain cost breaks that assumption. It says: not all cells/moves cost the same to traverse. Picture the drone flying over a real environment:

- Moving over open, flat ground → cheap (cost = 1, say)
- Moving over a hill, or against headwind → more expensive (cost = 3, maybe)
- Moving through some kind of restricted/high zone → very expensive (cost = 5)

So each cell (or each edge between cells) gets a **weight** representing how costly it is for the drone to *move there* — *not* the machine's battery drain, time, or risk.

This is a Reply to Claude...

total of steps y.

+ Sonnet 4.6

🔍📶🔊🔇

👍👎👏👊

rig nany

22:41 Sat 20 Jun🔍📶🔊🔇

So each cell (or each edge between cells) gets a **weight** representing how costly it is for the drone to move there — maybe modeling battery drain, time, or risk.

This is exactly why the path-cost function $g(n)$ exists and matters: $g(n)$ is the *running total* of terrain costs accumulated from the start to node *n* — not just a count of how many steps you've taken.

And here's the connection back to your earlier insight about BFS: once terrain costs are unequal, **BFS no longer guarantees the optimal (cheapest) path** — it still guarantees the path with the *fewest moves*, but fewest moves and lowest total cost are no longer the same thing. A 2-step path through expensive terrain could cost more than a 4-step path through cheap terrain.

Quick check: given that, which algorithm do you think you'd need if you want the drone to find the path that minimizes *total terrain cost* rather than just minimizing the number of moves — and does it need a heuristic, or can it work with uninformed search alone?

👍👎👏👊

maybe an A*

Good instinct that A* fits here — but let's slow down and tease apart two separate questions, because there's a more basic algorithm that handles "minimize total cost" *without* needing a heuristic at all.

Think back to the EXPAND discussion: a node's $g(n)$ is its accumulated cost from the start. What if an algorithm just always picked, from the frontier, whichever node has the **lowest $g(n)$ so far** — using a priority queue instead of a plain stack/queue?

That algorithm is called **Uniform-Cost Search (UCS)**. It's still classified as *uninformed* — it has no heuristic sense of where the goal is — it just always expands the cheapest-so-far node first. And it *does* guarantee the optimal (minimum total cost) path, even with unequal terrain costs.

So here's the real distinction to hold onto:

- **UCS** uses priority = $g(n)$ only (cost so far). Uninformed, optimal for cost.
- **Greedy Best-First Search** uses priority = $h(n)$ only (estimated cost to goal — the heuristic). Informed, but *not* guaranteed optimal.
- **A*** uses priority = $g(n) + h(n)$ (cost so far **plus** estimated cost remaining). Informed, *and* guaranteed optimal (if the heuristic has a certain property).

So A* isn't just "the cost-aware one" — it's specifically the one that **combines** real accumulated cost with an estimate of remaining cost. That combination is what makes it both efficient (like Greedy, it's guided toward the goal) and optimal (like UCS, it accounts for true cost).

Given that breakdown — for your drone lab, what heuristic do you think would make sense for $h(n)$ on a 2D grid, estimating distance from a cell to the target cell? (Hint: you may have used something like this in your linear algebra/numpy work — distance formulas.)

👍👎👏👊

so need the terrain cost for the weighted search

Exactly — that's the right instinct, but let's be precise about *which* algorithms actually need it, because not all of them do.

Terrain cost matters only when an algorithm's decision rule actually looks at cost. Let's go through your full list:

- **BFS, DFS, DLS, IDS** — none of these look at cost at all. They only care about the *structure* (which nodes are reachable, in what order via queue/stack). So even if your grid has terrain costs defined, these four algorithms would completely ignore them. They'd still run, but "ignoring terrain cost" means they're optimizing for *fewest moves*, not *cheapest* path.
- **UCS** — needs terrain cost. Its whole priority rule is "expand lowest $g(n)$," and $g(n)$ is built from terrain costs accumulated along the path. Without terrain cost (i.e., if everything cost 1), UCS degenerates into being identical to BFS.
- **Greedy Best-First Search** — technically doesn't *need* terrain cost, because its priority is purely $h(n)$, the heuristic estimate to the goal. It can ignore $g(n)$ entirely. (Though some implementations still track it for reporting purposes.)
- **A*** — needs terrain cost, because its priority is $g(n) + h(n)$, and $g(n)$ is exactly that running total of terrain costs.

So the real picture: **terrain cost is data that lives on the grid** (e.g., a cost value per cell or per edge), and it's only meaningful to the algorithms whose priority function actually references $g(n)$ — which is UCS and A*.

Quick question to check this lands: if your lab grid assigns different terrain costs to different cells, and you can plain BFS on it, would BFS still find a *valid* path to the goal? Would that path be the *cheapest* one in terms of total terrain cost?

👍👎👏👊

could you explain how a weighted grid problem in plementation would work

Let's build this up structurally rather than jumping to code, so you understand *why* each piece exists — then if you want, we can talk implementation specifics after.

1. Representing the grid with weights

Instead of just knowing "this cell exists / is blocked," each cell (or each move into a cell) needs an associated cost. Two common ways to model this:

- **Cell-cost model**: every cell has a cost value — e.g., `grid[row][col] = 3` means "it costs 3 to move into this cell." This is intuitive for terrain (e.g., grass = 1, mud = 3, water = *S*/impassable).
- **Edge-cost model**: the cost is attached to the *move itself* rather than the cell — e.g., moving from (1,1) to (1,2) costs *X*. This matters if cost depends on direction (e.g., moving uphill costs more than downhill).

For a drone terrain lab, the cell-cost model is usually simpler and probably what's intended — does your lab spec describe costs as belonging to *cells* or to *moves/edges*? That will shape the data structure.

2. The node's job changes

Recall from EXPAND: every node needs $g(n)$ — cost from start to this node. In the unweighted case, $g(child) = g(parent) + 1$. In the weighted case:

Code

```
g(child) = g(parent) + cost(child) [cell-cost model]
```

So each node object/tuple needs to store: state (coordinates), parent reference, and $g(n)$.

3. The frontier becomes a priority queue

This is the big structural shift from BFS/DFS. Instead of "add to the back of a queue" or "push onto a stack," you need a data structure that **always gives you the node with the lowest priority value** — this is what a **min-heap / priority queue** is for.

- For UCS: priority = $g(n)$
- For A*: priority = $g(n) + h(n)$

4. Why does this matter for correctness — the "re-relaxation" problem

Here's a subtlety that trips people up: with BFS/DFS, once you visit a node, you never need to revisit it — the first time you reach it is "good enough" because all costs are equal. But with weighted costs, **the first time you reach a node is not necessarily via the cheapest path**. You might find a cheaper route to a node *later* in the search.

So the visited-tracking logic needs to change from "have I seen this state before? if so, skip" to something more careful: "have I reached this state with a lower cost before? If this new path is cheaper, update it."

Before I go further into the implementation mechanics of point 4 — does that "a node might get a better/cheaper path discovered later" idea make sense given what you know about $g(n)$? Want to walk through a tiny concrete example of that happening on a grid?

👍👎👏👊

yes

Let's build a tiny concrete grid where this actually happens, so it's not abstract.

Grid setup (just a few cells):

Code

```
col0 col1 col2
row0 (0,0) (0,1) (0,2)
row1 (1,0) (1,1) (1,2)
```

Start = (0,0). We want to reach (1,1).

Let's say moving *into* a cell costs whatever that cell's terrain cost is:

- (0,1) costs **1** to enter
- (1,0) costs **5** to enter (rough terrain)
- (1,1) costs **1** to enter
- (0,2), (1,2) — ignore for now, not relevant to this example

Moves allowed: right and down only, for simplicity.

There are two possible paths from (0,0) to (1,1):

- **Path A:** (0,0) → (0,1) → (1,1)
- **Path B:** (0,0) → (1,0) → (1,1)

Let's see a Reply to Claude...

1. Sta

+ Sonnet 4.6

🔍📶🔊🔇

👍👎👏👊

1. Sta

2. Prio

22:41 Sat 20 Jun

That's true but let's get more specific — why would it actually break something if we didn't?

Picture this: the frontier currently has $[(1, 1), (2, 1)]$, and both of these nodes happen to share a common neighbor — say $(2, 2)$ is reachable from both.

If we **only** marked nodes visited when we **pop** them (not when we add them):

1. Pop $(1, 1)$ from front. Expand it. $(2, 2)$ is a neighbor, not yet visited $\rightarrow$ add it to frontier, mark visited. Frontier: $[(2, 1), (2, 2)]$

2. Pop $(2, 1)$ from front. Expand it. $(2, 2)$ is also a neighbor of $(2, 1)$ — but wait, if we only mark visited on *pop*, has $(2, 2)$ been marked visited yet? It hasn't been popped yet, only added.

So $(2, 1)$ would *also* add $(2, 2)$ to the frontier — a duplicate! Frontier: $[(2, 2), (2, 2)]$

Now you've got the same node sitting in the queue twice. It's not catastrophe (BFS would still work, just wastefully), but it gets worse with bigger grids — the frontier balloons with duplicates, and you do redundant expansion work for the same node multiple times.

So marking visited **at the moment of adding to frontier** (not at pop time) prevents this duplication — once a node is *scheduled* to be visited, we don't schedule it again, even if another path reaches it first.

Does that distinction — "visited at add-time vs visited at pop-time" — make sense as the actual mechanical reason, rather than just "so it's not visited again"?

yes, now what about greedy bfs

Good — let's mutate the BFS skeleton into Greedy Best-First Search, and I want you to spot the changes yourself before I confirm.

Here's BFS again, side by side with a Greedy BFS skeleton that has some blanks:

python

BFS (for reference)
from collections import deque

def bfs(grid, start, goal):
 frontier = deque([start])
 came_from = {start: None}
 visited = {start}
 while frontier:
 current = frontier.popleft()
 if current == goal:
 return reconstruct_path(came_from, start, goal)
 for neighbor in get_neighbors(current, grid):
 if neighbor not in visited:
 visited.add(neighbor)
 came_from[neighbor] = current
 frontier.append(neighbor)
 return None

Python

Greedy Best-First Search (some blanks for you)
import heapq

def greedy_bfs(grid, start, goal):
 frontier = [] # priority queue
 heapq.heappush(frontier, (_____, start)) # <-- BLANK 1: what's the priority
 came_from = {start: None}
 visited = {start}
 while frontier:
 current = heapq.heappop(frontier) # pops lowest priority
 if current == goal:
 return reconstruct_path(came_from, start, goal)
 for neighbor in get_neighbors(current, grid):
 if neighbor not in visited:
 visited.add(neighbor)
 came_from[neighbor] = current
 heapq.heappush(frontier, (_____, neighbor)) # <-- BLANK 2
 return None

Two things to notice structurally before you answer:

- The frontier changed from *deque* to a list managed by *heapq* (priority queue) — that's the data structure swap we discussed.
- Everything else (*visited-tracking*, *came_from*, *reconstruct_path*) stayed **identical** to BFS.

So the only real question is: **what goes in BLANK 1 and BLANK 2?** Given what you know about Greedy BFS's priority rule, what value should be pushed alongside each node?

heuristic for the start to goal and neighbour to goal

Exactly right. Let's make it concrete and precise.

Blank 1: when we push the *start* node initially, its priority is the heuristic estimate from *start* to *goal*:

python

heapq.heappush(frontier, (heuristic(start, goal), start))

Blank 2: same idea, but for each neighbor we're about to add:

python

heapq.heappush(frontier, (heuristic(neighbor, goal), neighbor))

Where *heuristic(a, b)* would be your Manhattan or Euclidean distance function from earlier — e.g.:

python

def heuristic(a, b):
 return abs(a[0] - b[0]) + abs(a[1] - b[1]) # Manhattan distance

So the only conceptual difference between BFS and Greedy BFS, in code, is:

1. deque $\rightarrow$ priority queue (heapq)
2. Priority value: none (just insertion order) $\rightarrow$ $h(n)$ (heuristic to goal)

Everything else — *visited tracking*, *came_from*, *path reconstruction* — is identical.

Here's an important thing to flag for your lab write-up, since you understood this earlier: Greedy BFS is **not guaranteed optimal**. It can get "flooded" by the heuristic — imagine a neighbor that looks great by straight-line distance to the goal, but is actually a dead end requiring a long detour around an obstacle. Greedy BFS will charge toward it anyway because it only looks at $h(n)$, completely ignoring how much it already cost to get there ($g(n)$).

Which leads directly into A*. Given the Greedy BFS skeleton above, and knowing A*'s priority is $g(n) + h(n)$ instead of just $h(n)$ — what do you think needs to change structurally? Specifically, what extra piece of information does each entry in the frontier need to carry now, that Greedy BFS didn't need?

can i use a priority queue for bfs

Interesting question — let's think about it from both directions: *can you mechanically do it, and should you*.

Mechanically — yes, you could. If you pushed every node onto a priority queue using, say, $g(n)$ (cost-so-far, which in unweighted BFS just means "number of steps taken") as the priority, you'd actually get... BFS! Lower $g(n)$ = fewer steps = explored earlier. This is actually exactly what **Uniform-Cost Search** is — and on a grid where every move costs 1, UCS and BFS produce *identical* exploration order, just through different mechanisms.

But here's the catch — efficiency. A *deque* gives you *popleft()*/*append()* in $O(1)$ time. A priority queue (*heap*) gives you *push/pop* in $O(\log n)$ time. On a small lab grid, you'd never notice the difference. But it's a meaningful inefficiency to introduce a heap when a queue does the *exact same job* for free, because in plain BFS, **insertion order** and **cost order** are **already the same thing** — there's no "priority" information being expressed that a queue doesn't already capture implicitly.

So the honest technical answer: using a priority queue for BFS works, but it's solving a problem that doesn't exist yet. It's like using a sorting algorithm to find the max of a list — correct, but doing more work than the structure of the problem requires.

Here's a question to check whether the reason behind this lands: if a grading rubric for your lab says "Implement BFS using the appropriate data structure for its frontier," do you think submitting a priority-queue version would likely lose you points — and if so, why, given that it produces the same *output*?

so in general out of all the search algorithms which one is more optimal and which is more efficient

Good question to pause on — but I want to flag something first: "more optimal" and "more efficient" aren't single rankings you can sort all six algorithms into. They depend on *what* you're optimizing for, and the answer changes depending on whether costs are uniform or weighted. Let's break it apart properly inste...

Optimality — "does it guarantee the best possible path?"

This is...

Algor

Reply to Claude...

+

Sonnet 4.6

path

22:41 Sat 20 Jun

Let's say UCS explores breadth-ish by cost. Walk through what happens:

1. Start $g(0,0) = 0$. Expand it. Children: $(0,1)$ with $g = 0 + 1 = 1$, $(1,0)$ with $g = 0 + 1 = 1$.

2. Priority queue now has $(0,1)$ at priority 1, $(1,0)$ at priority 1.

3. Pop the lowest priority — that's $(0,1)$, $g=1$. Expand it. Its child is $(1,1)$, reached via Path $A: (0,1) \rightarrow 1 + 1 = 2$.

4. Now the queue has: $(1,0)$ at priority 1, $(1,1)$ at priority 2 [reached via Path A].

5. Pop lowest priority — that's $(1,0)$, $g=2$. Is it the goal? Yes! Return this path: cost 2, via $(0,0) \rightarrow (0,1) \rightarrow (1,1)$.

Notice: $(1,0)$ never even got expanded — it was still sitting in the queue with priority 1 when we found the goal at cost 2 through a totally different route. UCS never had to "discover a cheaper path later" in this particular example, because it got lucky — the cheap route was found before the expensive one even got explored.

So let's now flip it — I want you to predict: what would need to be true about the *grid layout* (not the costs, the layout) for UCS to actually find a *worse* path to some node *first*, and *then* later discover a better one? Think about what kind of detour would cause that.

but could you explain manhattan distance and euclidean

Sure — let's connect this directly to the heuristic (h) role we touched on earlier, since these are the two modes for grid-based search.

Both answer the same question: **"roughly how far is this cell from the goal?"** — but they measure "distance" differently, based on what kind of movement is allowed.

Manhattan distance (a.k.a. taxicab distance)

This measures distance as if you can only move along grid lines — like a taxi driving through city blocks, never cutting diagonally through buildings. You have to count steps along rows and columns separately, then add them.

Formula, between point $(x1, y1)$ and $(x2, y2)$:

Code

$h = |x1 - x2| + |y1 - y2|$

Example: from $(1,1)$ to $(4,5)$:

- horizontal distance: $|4-1| = 3$
- vertical distance: $|5-1| = 4$
- Manhattan distance = $3 + 4 = 7$

This matches your drone setup well if the drone can **only move up/down/left/right** (no diagonals) — because that's exactly the kind of "city block" movement Manhattan distance assumes. It naturally estimates "at least this many moves are needed," since each move only changes one coordinate by 1.

Euclidean distance

This is straight-line, "as the crow flies" distance — the actual geometric distance, allowing movement in any direction, including diagonally.

Formula:

Code

$h = \sqrt{(x1 - x2)^2 + (y1 - y2)^2}$

Same example, $(1,1)$ to $(4,5)$:

- $h = \sqrt{(1-4)^2 + (1-5)^2} = \sqrt{9+16} = \sqrt{25} = 5$

Notice it's *smaller* than Manhattan distance (5 vs 7) — that makes sense, since a straight diagonal line is always shorter than or equal to a zigzag path along grid lines.

Why does the choice matter for your drone?

This connects to something important for A*'s optimality guarantee: the heuristic needs to be **admissible** — meaning it *never overestimates* the true cost to the goal. If your drone can only move in 4 directions (no diagonals), Manhattan distance is admissible (it never overestimates) but Euclidean distance is *not* admissible in that case — it would actually underestimate even more, since real movement can't cut diagonally.

If your drone can move diagonally (8 directions), which one do you think becomes the more "honest" / tighter estimate — Manhattan or Euclidean? Think about which one actually matches what a diagonal move "buys" you.

what is a priority queue

Good — let's build this from something you already know: queues and stacks.

You already understand a **queue** (FIFO) and a **stack** (LIFO) — in both cases, the order things come *out* is determined purely by the order they went *in*. No regard for "how important" or "how good" any individual item is.

A **priority queue** breaks that rule entirely. Instead of "first in, first out" or "last in, first out," it's: **"whatever has the best priority value, out first"** — regardless of when it was added.

So if you push items into a priority queue in this order:

Code

$push(A, priority=5)$
 $push(B, priority=1)$
 $push(C, priority=3)$

When you start popping, you get **B first** (priority 1 — assuming lower = better, which is the convention for cost-based search), even though it was added second. Then C (priority 3), then A (priority 5) — totally ignoring the insertion order.

This is exactly why UCS and A* need it: their whole logic is "always expand the node that looks most promising right now" — where "most promising" means lowest $g(n)$ for UCS, or lowest $g(n) + h(n)$ for A*. A plain queue or stack has no way to express "promising" — it only knows "old" vs "new." A priority queue is the data structure that lets the value *itself* (the cost/estimate) determine pop order.

Under the hood, priority queues are typically implemented with a **min-heap** — a tree-like structure that keeps the smallest element easily accessible at the root, and can *insert*/*remove* efficiently ($O(\log n)$) without needing to fully sort everything each time. In Python, this is usually *heapq*. In Java, *PriorityQueue*.

Quick check: given what you now know about priority queues — for A*, when you push a node onto the priority queue, what value do you think gets used as its priority? (You've actually already said this a few messages ago — want to restate it to make sure it's locked in?)

could you explain and help me with the implementation for a BFS, greedy BFS and A* search

Let's build these up one at a time — but I want you driving the logic, since you've already nailed the concepts (frontier behavior, $g(n)$, $h(n)$, priority queues). I'll set up the skeleton and ask you to fill in the key decision points, since that's where the actual learning happens for a lab like this.

Let's start with **BFS**, since it's the simplest, and use it to establish a pattern we'll reuse for the other two.

BFS structure (Python-ish pseudocode):

python

from collections import deque

def bfs(grid, start, goal):
 frontier = deque([start]) # queue: FIFO
 came_from = {start: None} # for reconstructing the path
 visited = {start}
 while frontier:
 current = frontier.popleft() # <-- pop from the FRONT
 if current == goal:
 return reconstruct_path(came_from, start, goal)
 for neighbor in get_neighbors(current, grid):
 if neighbor not in visited:
 visited.add(neighbor)
 came_from[neighbor] = current
 frontier.append(neighbor) # <-- add to the BACK
 return None # no path found

Before I move to Greedy/A*, I want to check you actually understand *every line*, because Greedy and A* are just small mutations of this same skeleton.

Two questions for you:

1. Why do we mark a node as **visited** the **moment we add it to the frontier**, rather than waiting until we pop and expand it? (Hint: think about what could go wrong with duplicate entries in the frontier if we didn't.)

2. `get_neighbors(current, grid)` is basically doing the job of **EXPAND** we discussed earlier — what do you think this function needs to check before returning a neighbor as valid? (Think grid boundaries and obstacles.)

we mark it as visited so that it is not visited again

That's true but let's get more specific — why would it actually break something if we didn't?

Picture...

share a

+

Sonnet 4.6

pen to

If we **only** marked nodes visited when we **pop** them (not when we add them):

